

Multiple Index Merge for Approximate Nearest Neighbor Search

Liuchang Jing
HKUST (GZ)

ljing248@connect.hkust-gz.edu.cn

Mingyu Yang
HKUST (GZ)

myang250@connect.hkust-gz.edu.cn

Lei Li
HKUST (GZ) & HKUST
thorli@ust.hk

Jianbin Qin
Shenzhen University
qinjianbin@szu.edu.cn

Wei Wang
HKUST(GZ) & HKUST
weiwcs@ust.hk

ABSTRACT

Approximate k nearest neighbor (AKNN) search in high-dimensional space is a foundational problem in vector databases with widespread applications. Among the numerous AKNN indexes, Proximity Graph-based indexes achieve state-of-the-art search efficiency across various benchmarks. However, their extensive distance computations of high-dimensional vectors lead to slow construction and substantial memory overhead. The limited memory capacity often prevents building the entire index at once when handling large-scale datasets. A common practice is to build multiple sub-indexes separately. However, directly searching on these separated indexes severely compromises search efficiency, as queries cannot leverage cross-graph connections. Therefore, efficient graph index merging is crucial for multi-index searching. In this paper, we focus on efficient two-index merging and the merge order of multiple indexes for AKNN search. To achieve this, we propose a reverse neighbor sliding merge (RNSM) that exploits structural information to boost merging efficiency. We further investigate merge order selection (MOS) to reduce the merging cost by eliminating redundant merge operations. Experiments show that our approach yields up to a $5.48\times$ speedup over existing index merge methods and $9.92\times$ speedup over index reconstruction, while maintaining expected superior search performance. Moreover, our method scales efficiently to 100 million vectors with 50 partitions, maintaining consistent speedups.

PVLDB Reference Format:

Liuchang Jing, Mingyu Yang, Lei Li, Jianbin Qin, and Wei Wang. Multiple Index Merge for Approximate Nearest Neighbor Search. PVLDB, 14(1): XXX-XXX, 2020.

doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

The k nearest neighbor (KNN) search in high-dimensional space is a fundamental problem that has received widespread attention

due to its extensive applications in data mining [10], information retrieval [38, 45], recommendation systems [11, 47], and the Retrieval-Augmented Generation (RAG) of Large Language Models (LLMs) [34]. However, the cost of exact KNN search is extremely high for large-scale datasets [28]. Therefore, extensive research has focused on studying approximate k nearest neighbor (AKNN) search [1–3, 5, 7, 8, 12, 13, 16–26, 30, 35, 37, 39, 40, 42, 44, 46, 48, 51–54, 64, 69, 71], trading accuracy for search efficiency. Among various AKNN methods, graph-based indexes are the state-of-the-art solution, demonstrating superior performance compared to other approaches [4, 14, 36, 60]. Graph-based methods represent each vector in the dataset as a node in a graph, where edges connecting nodes reflect proximity relationships designed to facilitate efficient navigation during search. Most graph-based methods construct a proximity graph (PG) through one of two approaches: (1) refining a random or approximate KNN graph [17, 18, 44, 72], or (2) incrementally inserting nodes into an empty graph [39, 40]. However, both indexing methods incur substantial memory and time costs for graph construction [4, 60], making it challenging for a single machine to build an index for large-scale datasets.

To avoid the prohibitively high space and time overhead of directly building a complete index, existing methods typically partition the data and build multiple sub-indexes. For example, disk-based indexing approaches [51, 59, 74] reduce memory peaks by constructing multiple sub-indexes, while distributed systems [49, 67, 73, 77] leverage multiple machines to improve construction efficiency. Despite the advantages of low memory peaks and distributed scalability, building multiple sub-indexes incurs a significant search performance degradation. This phenomenon is intuitive because searching m separate graph indexes independently requires traversing each from its entry point. Given that graph search complexity is sublinear with respect to dataset size [18, 35, 44], querying m partitions scales the computational cost by a factor of approximately m . We also evaluate it experimentally and present the result in Fig. 1. The search efficiency of the partitioned index is only half that of the complete index with 10 partitions, and this efficiency gap widens to nearly $1/6$ with 50 partitions. To address this performance degradation, recent approaches [51, 66, 68, 74] have been proposed, with the core idea being to connect separate graphs to enable navigation across partitions, therefore achieve the merge of indexes.

We categorize existing index merge methods into two families based on their strategies for connecting multiple sub-graph indexes: (1) **Overlap**-based [51, 68, 74] methods partition the dataset into multiple overlapping subsets for sub-graph index construction. Then, separate graphs are connected by concatenating the neighbors of overlapping nodes. (2) **Search**-based [40, 66] treat the source

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

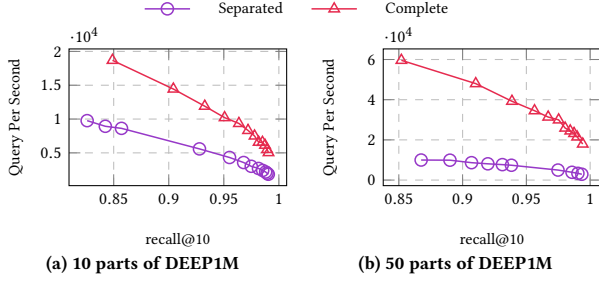


Figure 1: Separated indexed search performance. We randomly partitioned the DEEP1M dataset into 10 and 50 equally sized partitions for the sub-indices. During the search, we sequentially search all partitions and merge the results. We found that (i) searching with a split index has a significant performance gap compared with the complete index, and (ii) this gap widens as the number of partitions increases.

index data as queries to search in the target index. The neighbors of the subgraph index node are updated with the search results, which can be viewed as a stream insert. Thus, achieve the same proximity and navigation property of a complete graph index. The above two methods are widely used in industry. The overlap-based method is integrated into disk solutions such as DiskANN [9, 51] and Milvus [57], where each node is forced to be placed into two different partitions, and cross-partition connections are achieved through edge composites of that node. Systems built on LSM-tree-like organizations (e.g., BlendHouse [43] and VSAG [81]) continuously compact multiple small segments into larger ones in the background. During compaction, each resulting segment is assigned a freshly built per-segment graph index, and older per-segment indexes are discarded. This can be achieved through search-based index merging, but merging multiple small graphs makes its cost almost the same as rebuilding. Overall, existing overlap- and search-based approaches mainly reduce memory peak or support efficient write based on the LSM tree, but do not substantially reduce the computation cost of index construction. This motivates us to design new methods that explicitly accelerate the index merging process itself.

State-of-the-art. While the aforementioned methods focus on index merging scenarios, a recent study, SimJoin [66], tackles a related but different problem: approximate similarity join, i.e., identifying all pairs of vectors from two datasets whose distance is below a specified threshold. Although SimJoin is not specifically designed for index merging, its join-window sliding mechanism—which leverages the join search results of processed points to accelerate the joins for other points—can be applied to accelerating index merging. SimJoin organizes the two vector datasets into two graphs: a Minimum Spanning Tree (MST) based on vector distance and a pre-built proximity graph (PG). Join results are obtained by performing AKNN searches in the PG, and the search results of one node are reused by its adjacent nodes in the MST as starting points to facilitate their searches. However, this iterative reuse mechanism is not directly suitable for our setting, due to the following limitations when applied to index merging: (1) SimJoin does not natively support the merging of existing PG indexes, as it is tailored to pairwise joins over two raw datasets rather than index structures; (2) The sequential join-window sliding scheme introduces

strong data dependencies, which severely limit parallel scalability; and (3) The problem studied in SimJoin inherently designed for two-dataset joins and cannot be straightforwardly extended to multi-index merging.

Our Solution. In this paper, we propose a novel search-based multi-index merging framework that addresses the above limitations through two key techniques: *reverse neighbor sliding merge* (RNSM) and *merge order selection* (MOS). We first focus on efficiently merging two graph indexes and develop RNSM, a parallel-friendly index merging method with key innovations by leveraging previous search results to accelerate query processing. RNSM is built on three key components: *Neighbor Expand*, *Pivot Selection*, and *Sliding Merge*. RNSM explicitly exploits the structural information of both PG indexes via neighbor expansion. We then greedily select a set of *pivots* from the source index. By organizing the pivots and their neighbor, RNSM performs parallel-friendly local sliding via the reuse process of pivot search results, substantially improving scalability and merge efficiency. We further introduce MOS to optimize the merge order across multiple graph indexes. We base MOS on two key observations: (1) naive pairwise merging introduces redundant operations, and (2) the connectivity pattern between graph indexes strongly influences the search performance of the final merged index. With the above key observations, we design several MOS strategies tailored to different partition distributions to compute sparse merge order graphs that (i) reduce unnecessary merge operations and (ii) preserve or even enhance query efficiency in the final merged index.

Contribution. We summarize our main contributions as follows: *Problem Analysis and Motivation* (§ 3). We explored the key factors in index merging and found that nearest neighbor connections are crucial, thus transforming the index merging problem into a search optimization problem. We also analyzed the patterns of multi-index merging, providing new insights into the merge order selection.

Efficient Two-Index Merge Method (§ 4). We propose RNSM, an efficient two-index merge method that leverages structural information from both the source and target indexes to accelerate merging. RNSM identifies pivots in the source index by constructing approximate reverse nearest neighbor relationships. The search results of each pivot are then propagated to the pivot-neighbor to perform efficient local sliding.

Merge Order Selection (§ 5). We propose MOS, which formulates merge order selection as a graph optimization problem with dual objectives: minimizing merge cost through sparsity maximization and preserving search quality through diameter control. We introduce tailored strategies for both random and cluster-based partitions for different application scenarios.

Extensive Experiment Analysis (§ 6). We conduct comprehensive experiments on real-world datasets for multi-index merging. We integrate our framework into popular graph-based indexes, including HNSW, NSG, SSG, and τ -MNG, demonstrating its broad applicability. Our method delivers up to 2.65 $\times$ speedup while preserving search quality and efficiency. Moreover, the speedup ratio remains stable as the number of partitions increases with 50 sub-indices and 100 million data, indicating robust scalability.

Due to space constraints, some proofs and experiments are omitted, which can be found in our technical report [32].

2 PRELIMINARY

In this section, we formally define the AKNN search problem in Section 2.1. Then, we investigate the search and index construct process of current graph-based methods in Section 2.2.

2.1 The AKNN Search Problem

Table 1 lists the frequently used notations and their definitions. Before formally defining the AKNN search, we first describe the KNN search problem.

Definition 2.1 (KNN search). Given a dataset X with n vectors in the D -dimensional Euclidean space $\mathbb{R}^D$, and a query vector $q \in \mathbb{R}^D$. Let $\delta(x, y)$ denote the distance between two D -dimensional vectors x and y . The KNN search retrieves k vectors in X with minimal distance to q by evaluating $\delta(x, q)$.

However, with the rapid growth of dataset dimensionality and scale, exact KNN search fails to meet the efficiency requirements, known as the curse of dimensionality [28]. Therefore, most efforts have focused on the AKNN search. The AKNN search allows most methods to trade a small loss in accuracy for a significant gain in efficiency. Recall@K is commonly used as a metric to evaluate the precision of the AKNN search algorithm. Recall@K is defined as the ratio of the true k nearest neighbors that appear in the k nearest results returned by the AKNN search algorithm, and it is computed as follows:

$$\text{Recall@K} = \frac{|\text{AKNN}(q) \cap \text{KNN}(q)|}{K} \quad (1)$$

where $\text{AKNN}(q)$ is the K nearest results returned by the AKNN search, and $\text{KNN}(q)$ is the actual KNN for a given q . Based on the above definition, numerous AKNN Search Indexes have been proposed with the aim of achieving higher recall at a lower cost.

2.2 AKNN Search Index

Most AKNN search algorithms find the approximate nearest neighbors via well-designed indexes, which can be divided into four categories: hash-based [13, 19, 19, 24, 48, 52–54, 63, 64], quantization-based [1, 2, 5, 20–23, 30, 69, 71], tree-based [7, 12, 16, 42], and graph-based [3, 8, 17, 18, 25, 26, 31, 37, 39, 40, 44, 50, 51, 56, 58, 80]. Graph-based AKNN search algorithms have gained prominence due to their state-of-the-art search efficiency. Graph-based AKNN indexes take each vector in X as a node on the graph and construct a proximity graph (PG) to represent the similarity relationships between vectors. Specifically, given a dataset X with n vectors in the D -dimensional Euclidean space $\mathbb{R}^D$. $G = (V, E)$ denotes a graph index for X with vertex set V and edge set E . Every $v \in V$ corresponds to a unique vector x in X . An edge $(u, v) \in E$ denotes the neighbor relationship between $u, v \in V$.

Most graph-based methods imply a beam search algorithm presented in Algorithm 1 to retrieve AKNNs. During the beam search, set C is maintained as the candidate nearest neighbors of q . C is the ef nearest neighbors retrieved so far (line 1). The beam search starts from the given entry points in the graph (line 2). At each step of the search, it checks the neighborhood of the nearest unexpanded node

Algorithm 1: AKNN Beam Search

Input: Graph index $G = (V, E)$, entry points $\{ep\}$, query q , beam size ef, k

Output: Top- k approximate nearest neighbors of q

```

1 Initialize priority queue  $C = \emptyset$ ;
2  $C \leftarrow \{(ep, \delta(ep, q))\}$ ;
3 Each node  $v \in V$  set  $v.\text{expanded} \leftarrow \text{false}$ ;
4 while  $C$  has unexpanded nodes do
5    $u \leftarrow$  nearest unexpanded node in  $C$ ;
6    $u.\text{expanded} \leftarrow \text{true}$ ;
7   for each unexpanded neighbor  $v$  of  $u$  do
8      $C \leftarrow C \cup \{(v, \delta(v, q))\}$ ;
9   if  $|C| > ef$  then
10     $\text{resize } C$  to size  $ef$ ;
11 return  $k$  nearest nodes in  $C$ ;
```

Table 1: A Summary of Notations

Notation	Description
X_i	A set of points/vectors
D	The dimensionality of X
x, y	A data point/vector
$\mathbb{R}^d$	d -dimensional Euclidean space
$\delta(x, y)$	Euclidean/ L_2 distance between x and y
$G_i = (V_i, E_i)$	The graph index for X_i with vertex set V_i and edge set E_i
$\mathcal{L}(p, q)$	The local sliding cost from point p to q
$N(v, G)$	The neighbors of v in G
$N_k(x, X)$	The k -nearest neighbors of x in X
$R_k(x, X)$	The reverse k -nearest neighbors of x in X

and tries to insert these neighbors into C (lines 4-10). The algorithm terminates when no unexpanded node can be found in C (line 11). Graph-based methods construct PG indexes through a two-stage framework: (1) Perform beam search in Algorithm 1 on the current index and obtain neighbor candidates W from top- k results [39, 40] or visited nodes during searching [18, 44, 51]. (2) Apply the neighbor pruning rule of RNG [42, 55] or its variants [17, 35, 44, 46, 51, 72] to W to select neighbors. Reverse edges are further added for better connectivity.

3 PROBLEM ANALYSIS AND MOTIVATION

In this section, we formalize the multi-index merging problem and present key observations that motivate our method (Section 3.1). Based on these insights, we introduce the pipeline of our merging framework (Section 3.2).

3.1 Problem Analysis and Motivation

We begin our analysis by defining the m -graph index merge problem. Given m partitions $\{X_1, X_2, \dots, X_m\}$ of dataset X such that $X = \bigcup_{i=1}^m X_i$ and $X_i \cap X_j = \emptyset$ for all $1 \leq i < j \leq m$, the m -graph index merge problem is defined as follows:

Definition. Given m well-constructed graph indexes $\{G_1, G_2, \dots, G_m\}$, where $G_i = (V_i, E_i)$ and $V_i = X_i$, the m -graph index merge problem aims to construct a unified graph index $G = (V, E)$ that satisfies the following constraints:

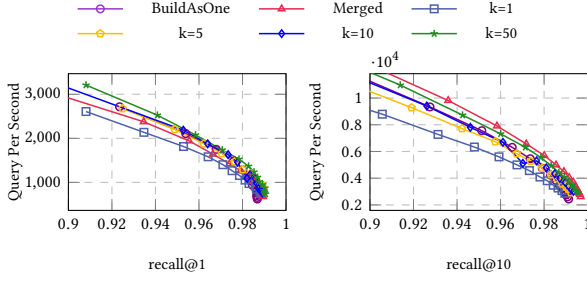


Figure 2: Nearest Neighbors being Critical Factor

- *Search Quality*: The merged graph G achieves comparable search performance to an index built from scratch on X .
- *Computational Efficiency*: The merging of multiple indexes requires less time than reconstructing an index for X from scratch.

We then discuss two main observations that motivate our proposed approaches to meet the above two requirements.

Key Contributing Components of Merging. We start by identifying the critical factors that preserve superior search performance during merging. As mentioned above, most graph-based methods construct proximity graphs using a two-stage framework, where higher-quality pruning candidates W lead to better index quality. These pruning candidates W aim to cover a large region around a given vertex, which directly leads to a high time cost. W can be divided into two parts: (1) Nearest neighbors that provide good connectivity among KNNs. (2) Distant neighbors that enable efficient graph traversal by introducing long-range edges and covering broader regions of the data space [74]. We find that when both indexes’ structural information is preserved in the merged index, **nearest neighbors become the critical factor for effective merging**, as distant neighbors are already captured within each sub-index.

EXAMPLE 1. As illustrated in Fig. 2, we investigate the impact of nearest neighbors from other indexes on merged index quality using NSG on the SIFT1M dataset. For $k=a$, pruning candidates W are constructed by concatenating existing neighbors in the source index with a actual nearest neighbors from the target index. **Rebuild** is an NSG index built directly on the complete SIFT1M dataset, while **Merged** is a merged index obtained by pairwise searching with $ef = 200$. Incorporating nearest neighbors from other indexes dramatically improves performance even with just $k = 1$, and it achieves the identical performance of **Rebuild** at high recall (≥ 0.99). With larger k values, the merged index achieves a better efficiency-accuracy tradeoff for AKNN search.

Based on this observation, we can safely reduce the merging problem to a more efficient formulation. Standard index construction typically requires large pruning candidate sets with $|W| \geq 200$. In contrast, for merging purposes, we only need to identify the top- k nearest neighbors from the target index for each vertex. Here, k can be substantially smaller, e.g., $k \in [1, 50]$. This reduction yields significant computational savings by limiting the search to a small

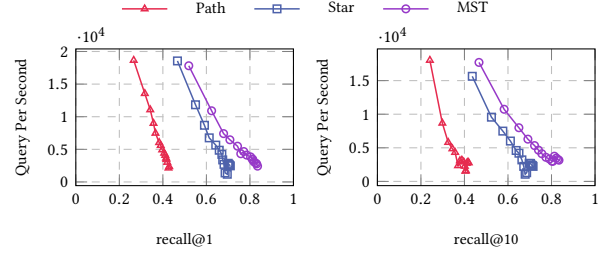


Figure 3: Search Performance of Different Merge Order

number of nearest neighbors from other indexes, making the merging process substantially more efficient without sacrificing search quality.

Merge Order Analysis. We then consider different merge orders for merging multiple indexes, where the merge order specifies which pairs of indexes to merge. According to our analysis, retrieving KNNs from other indexes for each vertex is critical, which naturally leads to a pairwise processing framework. However, this framework suffers from inherent computational redundancy for the following reasons: (1) KNNs from different partitions may occlude one another during merging. Empirically, only 9.2% of the 100-NNs remain as neighbors of the vertex after neighbor selection. (2) For cluster partitions, KNNs of a vertex are often concentrated in a subset of partitions rather than uniformly distributed across all partitions. Moreover, we observe that **even with the same number of merge operations, different merge orders result in significantly different search performance**.

EXAMPLE 2. We consider three different merge orders: **Path** constructs a linear merging chain, **Star** merges all indexes with a central one, and **MST** builds a minimum spanning tree based on inter-index similarity. As shown in Fig. 3, despite using an identical number of merge operations ($m - 1$ for m indexes), different merge orders yield dramatically different performance. **Path** fails to reach higher recall because KNNs of a vertex may be separated across partitions at opposite ends of the linear merge sequence. **Star** and **MST** significantly improve search performance through enhanced connectivity, enabling more comprehensive cross-partition neighbor discovery. However, **Star** imposes an imbalanced merge workload on the central hub index, resulting in inferior performance compared to **MST**.

3.2 Framework

We present the pipeline of our RNSM + MOS framework for efficient multi-index merging in Fig. 4, including three stages: sub-indexes construction, merge order selection, and sliding merge. We first prepare or receive sub-indexes for different partition types. Then, we use two MOS strategies for random partitions and cluster partitions to construct a sparse merge order graph for efficient multi-index merging. Finally, sliding merge accelerates the merge of two indexes via local sliding.

Sub-indexes construction. Our framework can be applied to different partition types without compromising performance. We first determine the partition type of the given indexes or data partitions. Then, we prepare the merge cost matrix for MOS based on the data distribution across partitions. For random partitions, the merge

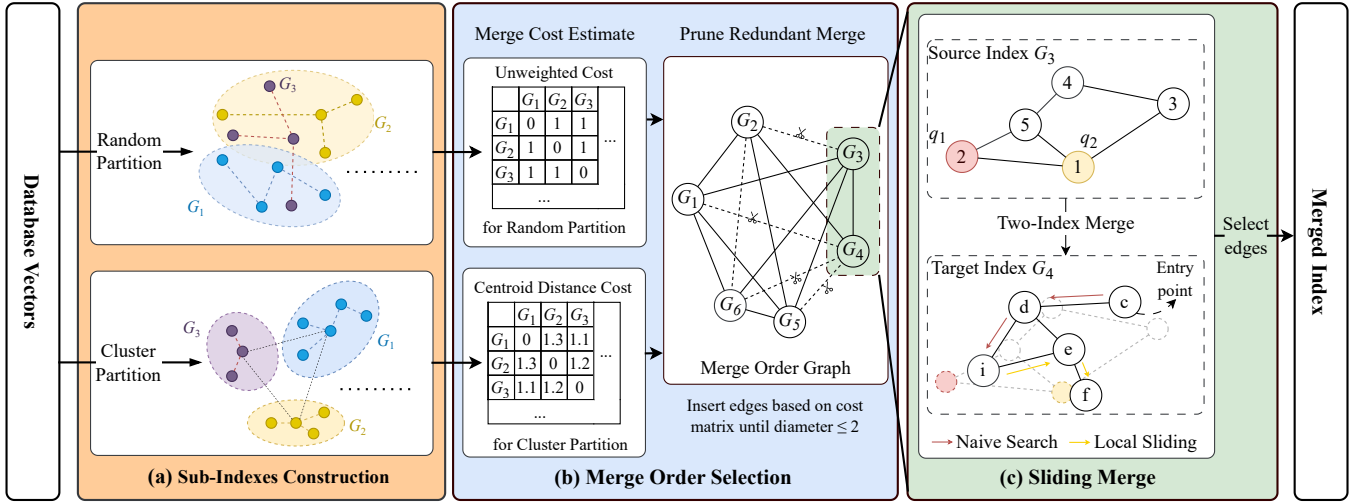


Figure 4: Pipeline of Our Multi-Index Merge Framework

costs for different index pairs are considered identical. For cluster partitions, we estimate the merge cost for two indexes using the distance between their centroids.

Merge Order Selection. MOS aims to reduce the cost of pairwise merging multiple indexes. Specifically, we model merge operations as a merge order graph and minimize the number of edges for efficient merging while maintaining sufficient connectivity for high-quality search on the merged index. We use the average degree of the merge order graph to measure merge time cost and the graph diameter to measure connectivity.

Sliding Merge. Finally, we merge the two adjacent indexes in the merge order graph using our RNSM algorithm. RNSM exploits prior search results to accelerate later searches. Specifically, we greedily select nodes with the maximum number of RNNs as pivots. Then, the pivots' search results in the target index are used as entry points to find cross-index neighbors for their RNNs.

4 THE EFFICIENT INDEX MERGING FRAMEWORK

This section details the proposed Reverse Neighbor Sliding Merge (RNSM) framework. We first formally define the concept of *sliding merge* and derive a corresponding cost model in Subsection 4.1 to quantify the merging overhead. Building on this foundation, we then introduce the RNSM algorithm in Subsection 4.2, demonstrating how it optimizes the merging process by minimizing the derived cost objective.

4.1 Sliding Merge Cost Model

Our preliminary analysis identifies the AKNN search within the target index as the primary bottleneck in index merging (see Fig. 2). Consequently, optimizing the search process on the target index is paramount. Prior research establishes that initializing graph-based searches with entry points topologically closer to the query target significantly reduces the search path length, thereby reducing latency [18, 51, 79]. Building on this insight, we propose *Sliding*

Merge, a strategy that leverages search results from neighboring nodes to accelerate processing. In this subsection, we formalize the cost model and define the optimization problem.

Notation and Sliding Semantics. Given a source graph index $G_1 = (V_1, E_1)$ and a target index $G_2 = (V_2, E_2)$, our objective is to merge G_1 into G_2 by performing an AKNN search in V_2 for every vector $x \in V_1$. The *Sliding Merge* framework employs two distinct search operations:

- (1) **Naive Search:** A standard AKNN search for query $x \in V_1$ starting from the default entry point of G_2 .
- (2) **Local Sliding:** A search for query x initialized using the result of a previous search for a pivot node p .

Let $\mathcal{L}(p, x)$ denote the search cost for query x . If a pivot p is used, the cost depends on the proximity between p and x ; otherwise, a fixed base cost is incurred.

Cost Metric. To ensure our model is hardware-agnostic, we employ the Number of Distance Computations (NDC) as the cost metric, a standard practice in nearest neighbor search analysis [21, 40, 51, 76, 81]. We define the cost of a *Naive Search* as a constant γ , derived from the average NDC of a search with a default entry point. For *Local Sliding*, recent findings [66] indicate that when a query x starts from a pivot p proximate in the high-dimensional space, the NDC required to converge follows a linear relationship with their distance $\delta(p, x)$ (see Fig. 6). Accordingly, we approximate the sliding cost as $\mathcal{L}(p, x) \approx \delta(p, x)$.

Problem Formulation. While local sliding offers lower costs than naive search ($\delta(p, x) < \gamma$ for nearby pivots), it necessitates a prerequisite: the pivot p must have already been searched. Ideally, every node would slide from its nearest resolved neighbor. However, identifying the optimal global pivot for x would itself require a costly search. To make this tractable, we exploit the topology of the source graph G_1 . Since G_1 is a proximity graph, neighbors in G_1 are inherently close in the vector space [18]. Thus, we restrict the candidate pivots for a node v to its immediate neighbors $N(v)$ in G_1 . This transforms the merge optimization into a pivot selection problem:

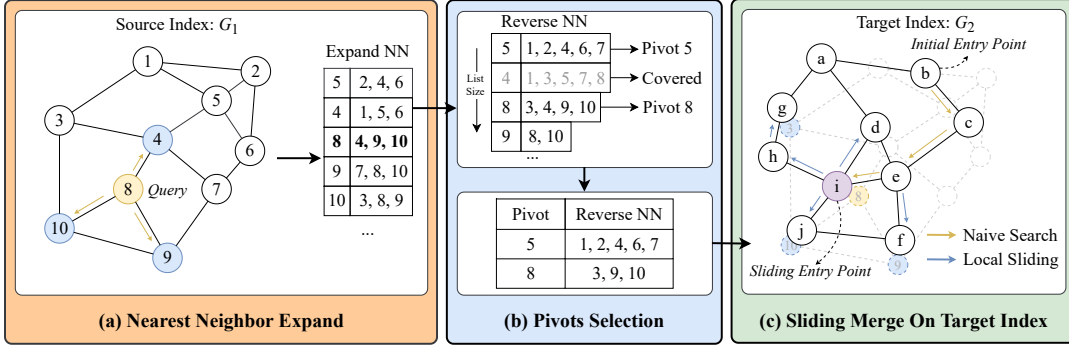


Figure 5: Example of Two-Index Merging with RNSM

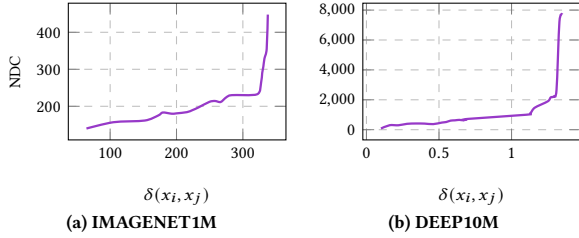


Figure 6: Verification of Distance Cost Model. In datasets IMAGENET1M and DEEP10M, the cost of sliding from x_i to x_j is linear to the distance when x_i and x_j are close.

we must select a subset of nodes to perform Naive Search (pivots), such that all other nodes can perform low-cost Local Sliding from a neighbor. We formally define this as follows:

Definition 4.1 (Minimum Cost Dominating Pivot Selection (DPS)).

- **Input:** A vector dataset X , a graph $G = (V, E)$ where node $v_i \in V$ corresponds to $x_i \in X$, a distance function $\delta(\cdot, \cdot)$, and a naive search cost γ .
- **Output:** A pivot subset $P \subseteq V$ that minimizes the total cost $\mathcal{L}(P)$, subject to the dominating set constraint.
- **Constraint:** Every node $v \in V$ must be covered:

$$v \in P \vee \exists p \in P \text{ s.t. } (v, p) \in E$$

- **Objective:** Minimize $\mathcal{L}(P) = \mathcal{L}_{naive}(P) + \mathcal{L}_{slide}(P)$, where:

$$\mathcal{L}_{naive}(P) = \gamma \cdot |P|$$

$$\mathcal{L}_{slide}(P) = \sum_{v \in V \setminus P} \min_{p \in P \cap N(v, G)} \delta(x_v, x_p)$$

THEOREM 4.2. *The Minimum Cost Dominating Pivot Selection (DPS) problem is NP-hard.*

PROOF SKETCH. We prove the NP-hardness of DPS via reduction from the Minimum Dominating Set (MDS) problem [33]. Consider an instance of DPS where γ is sufficiently large such that $\gamma > \sum_{v \in V} \max_{u \in N(v)} \delta(v, u)$. In this scenario, the cost is dominated by the number of pivots $|P|$, rendering the sliding cost negligible. Minimizing $\mathcal{L}(P)$ becomes equivalent to minimizing $|P|$ subject to the domination constraint, which is exactly the MDS problem. For detailed proof, please refer to our technical report. $\square$

Given the intractability of the DPS problem, we proceed to investigate greedy heuristic algorithms. These heuristics must not only

Algorithm 2: Reverse Neighbor Sliding Merge (RNSM)

Input: Source Graph $G_1 = (V_1, E_1)$, Target Graph $G_2 = (V_2, E_2)$, expansion factor k^+ , reverse k

Output: Merged Graph $G = (V, E)$

```

1  $V \leftarrow V_1 \cup V_2; E \leftarrow E_1 \cup E_2;$ 
2  $P \leftarrow \emptyset;$  // Set of Pivots
3  $Covered \leftarrow \emptyset;$  // Set of merged nodes
/* Phase 1: Pre-computation */
4  $G'_1 \leftarrow \text{ExpandNeighbors}(G_1, k^+);$ 
5  $\{R_k(x) \mid \forall x \in V_1\} \leftarrow \text{BuildReverseIndex}(G'_1, k);$ 
6  $SortedNodes \leftarrow \text{Sort } V_1 \text{ by } |R_k(x)| \text{ descending};$ 
/* Phase 2: Pivot Selection */
7 for  $x \in SortedNodes$  do
8   if  $x \notin Covered$  then
9      $P \leftarrow P \cup \{x\};$ 
10     $TargetSet \leftarrow R_k(x) \setminus Covered;$ 
11    Assign  $TargetSet$  to slide from  $x$ ;
12     $Covered \leftarrow Covered \cup \{x\} \cup TargetSet;$ 
/* Phase 3: Parallel Merging */
13 for  $p \in P$  in parallel do
14   // Pivot: Naive Search with Default ef and k
15    $Res_p \leftarrow \text{SearchGraph}(G_2, \text{query} = p, \text{entry} = \text{default});$ 
16   UpdateGraph( $G, p, Res_p$ );
17   // Followers: Local Sliding
18   for  $y \in Assigned(p)$  do
19      $Res_y \leftarrow \text{SearchGraph}(G_2, \text{query} = y, \text{entry} = Res_p);$ 
20     UpdateGraph( $G, y, Res_y$ );
19 return  $G;$ 
```

approximate the optimal pivot selection but also accommodate the parallelization requirements of high-throughput index construction.

4.2 Reverse Neighbor Sliding Merge

While the DPS problem provides a theoretical objective, it is NP-hard. A standard approximation is the greedy strategy: iteratively selecting the pivot with the highest *unit value* (coverage nodes divided by cost). However, applying this directly to index merging is problematic for two reasons. First, *selection dependency*: the unit value of a candidate pivot changes whenever a neighbor is selected, requiring sequential updates that break parallel efficiency. Second,

limited visibility: relying solely on the sparse connections in the source PG graph limits the ability to find the true nearest pivot, potentially forcing sub-optimal sliding paths.

The RNSM Strategy. To overcome these limitations, we propose *Reverse Neighbor Sliding Merge (RNSM)*. The core intuition is to resolve pivot selection *globally* before merging, enabling full parallelism. Instead of dependent selection, we exploit the proximity properties of the Reverse k -Nearest Neighbor (RNN) graph. We find that if a node x appears in the KNN list of y , x is inherently a good pivot for y due to proximity. Consequently, nodes with a high-quality RNN graph are natural hubs that can serve as cost-efficient pivots for many neighbors. The RNSM approach consists of three phases, detailed in Algorithm 2:

- (1) **Neighbor Expansion (Line 4):** We first densify the source graph G_1 . Existing PGs are sparse approximations of the true KNN graph. We expand the neighborhood of each node x to a larger size k^+ by traversing the graph. This creates a richer set of candidate connections, ensuring that nodes can access their true nearest pivots, aligning with the DPS objective.
- (2) **Reverse Inversion (Line 5):** We construct the Reverse KNN set $R_k(x) = \{y \mid x \in N_k(y)\}$ for every node with a relative small k (i.e. 3, 5, 7).
- (3) **Pivot Selection (Lines 6-10):** We sort nodes descending by their reverse degree $|R_k(x)|$. We greedily select uncovered nodes as pivots. If x is selected as a pivot, all its unassigned reverse neighbors $R'_k(x)$ are assigned to slide from x .
- (4) **Parallel Execution (Lines 11-16):** Finally, we execute the merge in parallel. Pivots perform a *Naive Search* on G_2 . Their search results become the entry points (or candidate list) for the *Local Sliding* of their assigned slide neighbor.

EXAMPLE 3. As illustrated in Fig. 5, we merge node 8 into G_2 . Initially, with no prior search results available, we perform a naive search for node 8: we take node 8 as the query and search from the global entry point b in G_2 via Algorithm 1, retrieving the AKNN result i . Since node 4 is close to node 8 in G_1 , its AKNN in G_2 should be close to i as well. Exploiting this property, i serves as an effective local entry point. We then perform local sliding from i for node 4: we search for node 4 in G_2 starting from i , and obtain its AKNN j with significantly fewer hops. We propagate search results from the last iteration as local entry points until no nearby unmerged nodes remain, then restart with a naive search for another unmerged node.

4.3 Analysis

Advantages of RNSM. Using Reverse KNNs yields four structural benefits: (1) **Natural Hub Identification:** The metric $|R_k(x)|$ identifies density peaks (hubs) purely from graph topology without requiring expensive clustering or centroid computations. (2) **Dynamic Load Balancing:** Unlike fixed k approaches, RNSM allows variable cluster sizes. Dense regions generate pivots with large $|R_k(x)|$, amortizing the naive search cost over more nodes, while sparse regions degenerate gracefully. (3) **Cost Minimization:** By selecting pivots with the most extensive coverage first, we greedily minimize the number of required pivots $|P|$, directly optimizing the naive search term in our cost model. (4) **Proximity Guarantee:** A follower y is only assigned to pivot x if $x \in N_k(y)$. This guarantees

that the starting point of the sliding search is strictly within the k -nearest neighbors of the query, ensuring a low $\delta(x, y)$ and thus a minimal sliding cost.

Complexity Analysis. We analyze the time complexity of merging a source graph index G_1 into a target graph index G_2 to produce a unified index $G = (V, E)$, where $n = |V|$ denotes the total number of vertices. Without loss of generality, we assume the source index is not larger than the target index (i.e., $|G_1| \leq |G_2|$). If initially $|G_1| > |G_2|$, the two are swapped so that we always merge the smaller structure into the larger one—a common optimization that simplifies the analysis. A key property of these graph-based indices, established in prior work [18, 29], is that the maximum node degree is bounded by a constant. This sparsity guarantee is crucial for achieving an efficient merge procedure. The merging process consists of three sequential stages:

1. Neighbor Expansion: Expanding neighbors involves incremental search with the graph index. Start with the nearest neighbor; the incremental cost of searching the top- k' neighbors can be expressed as $O(k')$ with a small constant factor [70]. This efficiency stems from interpreting the existing proximity graph (PG) index as a compressed representation of the k -nearest neighbor graph [18]. *Total Complexity:* $O(n \cdot k^+)$.

2. Pivot Selection: Constructing the Reverse KNN (RNN) graph requires iterating over all edges in the expanded graph. Given that the total number of edges is $n \cdot k^+$, the inversion step takes $O(n \cdot k^+)$ time. Subsequently, sorting the nodes on the RNN graph using bucket sort can be performed in $O(|V| + |E|)$ time. Since $|V| = n$ and $|E| = n \cdot k < n \cdot k^+$, the *total complexity* of the entire process remains $O(n \cdot k^+)$.

3. Sliding Merge: Let $|P|$ be the number of selected pivots.

- **Pivots:** Perform naive AKNN search. Assuming a hierarchical graph structure (like HNSW), the search complexity is c_{naive} . Total: $O(|P| \cdot c_{naive})$.
- **Followers:** Perform local sliding. Since the start node is a neighbor, the search path is short. As derived in our cost model, this is proportional to the distance $\delta(p, y)$, which is bounded by a small constant c_{slide} in practice. Total: $O((n - |P|) \cdot c_{slide})$.

Overall Complexity: Both local sliding and naive search have a higher cost than neighbor expand, where $c_{naive} > c_{slide} > k^+$. The overall complexity can be derived to $O(|P|c_{naive} + (n - |P|)c_{slide})$. Since $|P| < n$ (typically 5% to 10% of nodes depend on k), the dominant term is reduced from $n \cdot c_{naive}$ (all-naive search) to roughly $n \cdot c_{slide}$, representing a significant speedup.

5 MERGE ORDER SELECTION FOR MULTIPLE INDEXES

In this section, we minimize the multi-index merge cost by reducing redundant operations. We first formalize the merge order selection task. Then, we propose algorithms for our formalized problem.

Merge Order Selection Recall that merging requires finding the KNNs in all other indexes for every node in each source index. Naive approach requires $O(m^2)$ index merge operations between m indexes. However, this approach contains significant redundancy in practice. Besides, our empirical analysis (Fig. 3) shows that different merge order strategies yield substantially different search

performance. Thus, we formally define the task as follows: Given m dataset partitions $\{P_1, P_2, \dots, P_m\}$, we represent partitions as vertices $V = \{v_1, \dots, v_m\}$ and merge operations as edges E , forming a merge order graph $\mathbb{G} = (V, E)$. Each edge $(v_i, v_j) \in E$ indicates that we perform index merging between index G_i and G_j . The merge order selection (MOS) task aims to adjust the merge order graph $\mathbb{G}$ to achieve better individual sub-index connections at low cost.

Cost Model. Given the notation above, our objective is to extract a subgraph from the complete graph of pairwise merges. We aim to identify a merge schedule—defined by this subgraph—that achieves search performance comparable to either a full index rebuild or an exhaustive pairwise merge strategy. For clarity, we analyze random and clustered partitions separately. For random partitions, we assume that merging any two partitions incurs a unit cost. For clustered partitions, the merge cost between two partitions depends on the distance between their centroids, as derived in § 4.1. Moreover, § 3.1 indicates that Path, Star, and MST-based merge strategies can lead to substantially different search performance. These differences can be understood through graph-theoretic design properties such as connectivity, diameter, and total edge weight. Unfortunately, relying on a single subgraph perspective did not achieve our expected superior search performance.

Problem. Therefore, we propose extracting subgraphs from multiple perspectives with minimal cost, specifically:

Definition 5.1 (Minimum Cost Merge Order Selection (MOS)).

- **Input:** Given a set of m vertices $V = \{v_1, v_2, \dots, v_m\}$ and a symmetric cost matrix C where C_{ij} represents the cost of connecting vertex v_i and v_j , diameter bound Δ and degree bound R .
- **Output:** An undirected graph $\mathbb{G} = (V, E)$ where $E \subseteq \binom{V}{2}$ such that:
- **Constraint:** (1) Connectivity: The graph $\mathbb{G}$ is connected. (2) Maximum Degree: For all vertices $v \in V$, the degree $\deg(v) \leq R$. (3) Graph Diameter: The diameter of $\mathbb{G}$ is at most Δ .
- **Cost Objective:** Minimize the total cost of the graph:

$$\min \sum_{(v_i, v_j) \in E} C_{ij}$$

The constraints serve distinct objectives:

- (1) **Connectivity:** This is the fundamental completeness requirement, ensuring that the search space is fully traversable.
- (2) **Degree Bound (R):** This constraint parallel-friendly design. The index merge operation locks a large proportion proximity graph node neighbors. A vertex with excessive degree serializes updates and causes the parallel workload imbalance.
- (3) **Diameter Bound (Δ):** This constraint governs the worst-case search latency. The diameter determines the maximum number of intermediate index hops required to traverse between any two partitions.

While minimizing the total edge weight reduces the direct merge costs, the graph edge weights also affect the runtime performance from our empirical analysis in § 4.3. Ignoring the diameter bound reduces the problem to the NP-hard Bounded Degree Minimum Spanning Tree Problem [33]. To address this NP-hard problem efficiently, we propose a greedy heuristic as detailed in Algorithm 3.

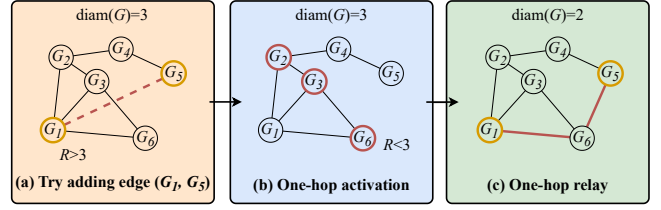


Figure 7: Example of Merge Order Selection

Algorithm 3: Merge Order Selection

Input: Partition Centroids $\{c_1, c_2, \dots, c_m\}$, Cost Matrix C , Degree Bound R , Diameter Bound Δ

Output: Merge order graph $\mathbb{G} = (V, E)$

```

1  $V \leftarrow \{c_1, c_2, \dots, c_m\}$ ;
2  $E \leftarrow \emptyset$ ;
3  $H \leftarrow$  distance matrix in hops, initialized to  $\infty$ ;
4  $Distances \leftarrow \emptyset$ ;
5 for  $c_i \in V$  do
6    $Distances[c_i] \leftarrow$  average distance of  $R$  nearest centroids;
7  $order \leftarrow V$  sorted in ascending order of  $Distances$ ;
8 for  $v \in order$  do
9   while has nearest vertex  $x$  with  $H[v][x] > \Delta$  do
10    if  $\deg(v) < R$  and  $\deg(x) < R$  then
11       $E \leftarrow E \cup (v, x)$ ;
12    else if  $\deg(v) < R$  and  $\deg(x) \geq R$  then
13       $x' \leftarrow \arg \min_{x' \in \text{Neighbor}(x), \deg(x') < R} C_{x', v}$ ;
14       $E \leftarrow E \cup (v, x')$ ;
15    else if  $\deg(v) \geq R$  and  $\deg(x) < R$  then
16       $v' \leftarrow \arg \min_{v' \in \text{Neighbor}(v), \deg(v') < R} C_{v', x}$ ;
17       $E \leftarrow E \cup (v', x)$ ;
18    Update  $H$ ;
19 return  $G$ ;
```

Algorithm. Algorithm 3 introduces the detailed procedure of MOS. We begin with an empty graph $\mathbb{G}$ and iteratively add edges (lines 1-3). We process vertices $v \in V$ in ascending order of their average distance to the R nearest centroids, prioritizing centrally-located partitions (lines 4-8). For each v , we iteratively connect it to its nearest neighbor x that is more than Δ hops away (line 9). If both v and x have degree less than R , we directly add edge (v, x) (lines 11-13). If exactly one vertex has reached degree limit R , we employ a 1-hop relay strategy (lines 14-21), select a neighbor x' of the degree-saturated vertex, and connect it to the unsaturated one. Note that the subgraph may not exist due to the Moore bound [27]; however, we still enforce connectivity in practice for result completeness.

EXAMPLE 4. As illustrated in Fig. 7, MOS constructs a workload-balanced merge order graph with diameter 2 by inserting and relaying edges within 1-hop neighborhoods. Suppose we insert edge (G_1, G_5) when the degree limit of 3 is exceeded for G_1 (i.e., $\deg(G_1) = 4$). We then relay this connection to one of the 1-hop neighbors of G_1 , and G_6 is selected since $\deg(G_6) < 3$. By relaying edge (G_1, G_5) as (G_5, G_6) , we obtain a graph with maximum degree at most 3 and diameter 2.

Complexity. We now analyze the time complexity of Algorithm 3. The ordering phase requires $O(m^2 \log R)$ for computing average distances of R nearest neighbors and sorting vertices. The main loop adds $O(m \cdot R)$ edges total, with each edge addition requiring $O(m \cdot R)$ time for BFS-based incremental hop-distance matrix updates, yielding a complexity of $O(m^2 R^2)$. Overall, the time complexity of our MOS algorithm is

$$O(m^2 R^2). \quad (2)$$

Despite the high complexity of updating the hop distance matrix, in practice, the number of partitions and the degree bound are relatively small. Even with around 100 partitions, the overall time cost of the MOS algorithm is only 1-2 seconds, which is negligible compared to the index construction time.

6 EXPERIMENT

6.1 Experiment Setting.

Datasets. We use six public datasets with diverse scales and types. Including datasets that are widely used in the benchmarks (SIFT, DEEP, GIST)¹ and datasets generated from the latest embedding models (IMAGE², ANTON³, MARCO⁴). The statistics of those datasets are summarized in Table 2, where *Base Size* denotes the dataset cardinality and *Query Size* denotes the number of queries for testing. We also take several slices of distinct sizes from the original datasets to test the scalability of our framework. For example, we use DEEP1M to represent 1 million slices of the DEEP dataset, and use ANTON10M to indicate the 10 million scale in the experiment.

Methods. We compare the following index merge methods:

- *Rebuild*: Rebuild the complete proximity graph directly.
- NM: Naive merge (NM) using naive search without local sliding.
- FGIM: Method for merging PG indexes and transforming it into a KNN graphs merging and refining problem [65].
- Overlap: Overlapping-based index merge method used in disk-based solution [51].
- RNSM: Our reverse neighbor sliding index merge(RNSM) method without merge order selection.
- RNSM*: Our reverse neighbor sliding index merge method with merge order selection for random partitions.
- RNSM+: Our reverse neighbor sliding index merge method with merge order selection for cluster partitions.

Performance Metrics. We evaluate the merged graph index based on two primary metrics: search performance and index construction (merge) efficiency.

- **Search Performance:** We measure efficiency via Queries Per Second (QPS) and accuracy via Recall@K previously defined in § 2. Consistent with standard practice, we report the average Recall@10 and QPS with the top 10 AKNN search over the query set.
- **Construction Efficiency:** We quantify efficiency using the speed-up ratio relative to the Naive Merge (NM), FGIM, rebuild

Table 2: Dataset Statistics

Dataset	Dimension	Base Size	Query Size	LID[6]
SIFT	128	1,000,000	10,000	19.4
DEEP	96	100,000,000	1,000	19.5
GIST	960	1000,000	1,000	38.9
IMAGE	768	10,000,000	10,000	21.2
ANTON	1024	10,000,000	1,000	28.9
MARCO	1024	10,000,000	1,000	31.0

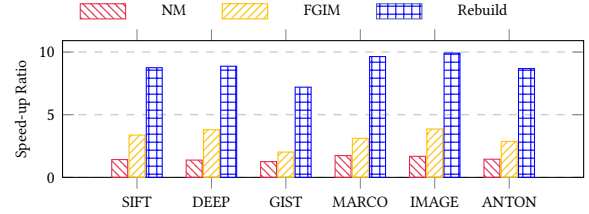


Figure 8: The Test of Index Merge Efficiency Speedup

index baselines, and the overlapping construction strategy employed by DiskANN.

Parameter Setting. We integrate our methods with various graph-based indexing algorithms, each with its own parameters. We leave the other parameters as default [61] and only consider the beam search queue size ef . In particular, we examine several widely adopted graph-based AKNN search algorithms: HNSW [40], NSG [18], SSG [17], and τ -MNG [44]. To determine the parameters k , k^+ , in Algorithm 2, we set the default beam size that achieves an average Recall@10 > 0.98 on separated indexes as ef based on observation of Fig. 2. Then, we employ grid search to determine the optimal parameter values of k^+ and k for Algorithm 2. For fair comparison, we set the same beam search parameter and max PG node degree used for NM, and RNSM. The parameter of FGIM is set according to its default settings [65].

Implementation Environment. All experiments were conducted on a server running Ubuntu 22.04 LTS, equipped with dual Intel Xeon Platinum 8370C CPUs (2.80GHz, 32 physical cores each, supporting AVX-512) and 512 GB of DRAM. The proposed framework and baselines were implemented in C++ based on the hnswlib library [41] and compiled using GCC 13.1.0 with -O3 optimization.

6.2 Experiments Results

Exp-1: Test of Indexes Merging Efficiency. We begin by evaluating the merging efficiency and subsequent search performance of our RNSM framework. For the setup, we randomly partition each dataset in Table 2 into two disjoint subsets of 0.5 million vectors and construct indexes using hnswlib. We compare RNSM against NM, FGIM, and a reconstruct-from-scratch baseline *Rebuild*. Fig. 8 reports the speed-up ratio of RNSM over these methods across six datasets. RNSM achieves significant efficiency gains over all baselines. For example, RNSM delivers speed-up ratios up to 1.74× over NM, 3.86× over FGIM, and 9.92× over *Rebuild*. Fig. 9 further demonstrates the search performance (Recall vs. QPS). RNSM achieves a trade-off comparable to existing methods and the *Rebuild* baseline

¹<https://www.cse.cuhk.edu.hk/systems/hash/gqr/datasets.html>

²https://huggingface.co/datasets/kinianlo/imagenet_embeddings

³<https://huggingface.co/datasets/anton-l/wiki-embed-mxbai-embed-large-v1>

⁴<https://huggingface.co/datasets/CoHere/msmarco-v2.1-embed-english-v3>

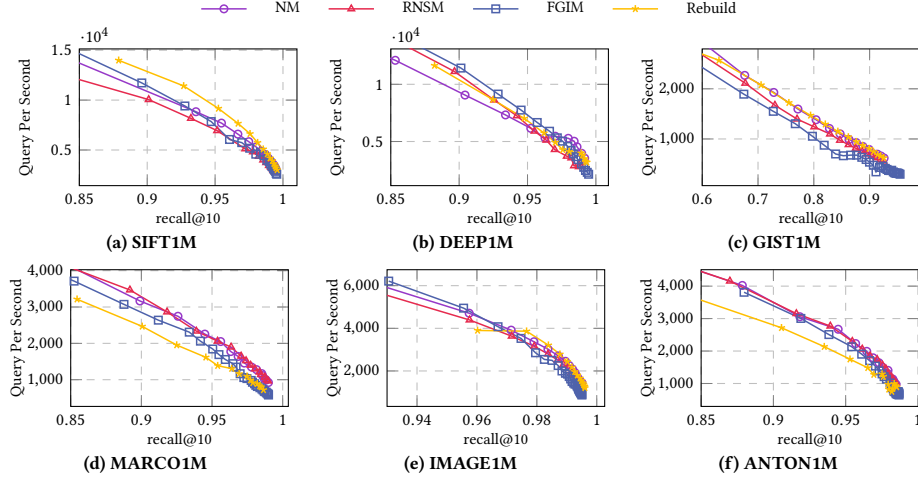


Figure 9: Recall@10-Query Per Second Tradeoff of Various Method

Table 3: Comparison of DiskANN and RNSM+. (Time: seconds, QPS_x: Query Per Second with recall@10=x)

Dataset	Maximum size: 5M						Maximum size: 2.5M					
	DiskANN			RNSM*			DiskANN			RNSM*		
	Time	QPS _{0.90}	QPS _{0.95}	Time	QPS _{0.90}	QPS _{0.95}	Time	QPS _{0.90}	QPS _{0.95}	Time	QPS _{0.90}	QPS _{0.95}
DEEP10M	1247	4538	3042	630	4554	3175	1206	5295	3225	767	4901	3493
ANTON10M	4116	1672	783	2928	1637	819	4084	1803	780	2649	1791	797
MARCO10M	4410	2021	1436	3138	2573	1751	4446	2308	1528	3295	2429	1662
IMAGE10M	2575	3509	2314	1518	3423	2395	3185	3603	2550	1582	3807	2456

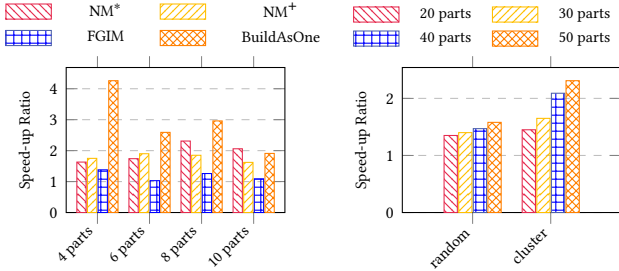
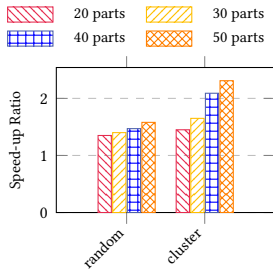


Figure 10: Speed-up Ratio Comparison across Different Partitions with MARCO10M

Figure 11: Scalability Study of Merge Efficiency on DEEP100M



on all datasets. These results confirm that our framework substantially improves merge efficiency without compromising the search performance of the resulting index.

Exp-2: Test of Multiple Indexes Merging. We further evaluate the search performance and merge efficiency when merging multiple indexes. We vary the number of partitions ($\{4, 6, 8, 10\}$) and assess robustness across different data distributions using both random (RNSM*) and cluster (RNSM+) partitioning. Experiments are conducted on DEEP10M, MARCO10M, IMAGE10M, and ANTON10M. Note that we compare RNSM* against both NM and FGIM, but compare RNSM+ only against NM, as FGIM is incompatible with cluster partitions. Fig. 10 reports the merge efficiency on MARCO10M (full results in technical report [32]). Our

methods achieve substantial speedups over baselines. For example, on MARCO10M, RNSM* delivers speed-up ratios of up to $2.31\times$ over NM, $1.38\times$ over FGIM, and $4.26\times$ over *Rebuild*, while RNSM+ achieves a $1.90\times$ speed-up over NM. Regarding search quality, as shown in Fig. 12, our framework maintains performance comparable to baselines across varying partition numbers and distributions.

Exp-3: Comparison with Overlap-based Method. We compare our framework against the overlapping partition strategy used in DiskANN on 10M-scale datasets. Unlike DiskANN, which partitions data into overlapping shards and builds sub-indexes from scratch, RNSM directly merges existing sub-indexes. For a fair comparison, we align the maximum shard size of Overlap with the total vector count of the indexes merged by RNSM (e.g., if Overlap uses a 5M shard size, RNSM merges four 2.5M sub-indexes). We also use the same index of Overlap strategy to focus on merge algorithm itself. We include both partitioning and construction time for both methods and report the QPS with 0.95 recall in Table 3. Our method achieves comparable search efficiency while significantly reducing index construction time. Specifically, RNSM is consistently faster across all datasets even without any pre-built indexes, delivering speedups of up to $2.01\times$ compared to the scratch-build approach of DiskANN. Furthermore, the index construction quality is better than the Overlap method, resulting in a higher QPS at 0.95 recall.

Exp-4: Scalability Study for Index Merging. We assess the scalability of our methods regarding dataset size and partition count on the large-scale DEEP100M dataset. We partition the data into

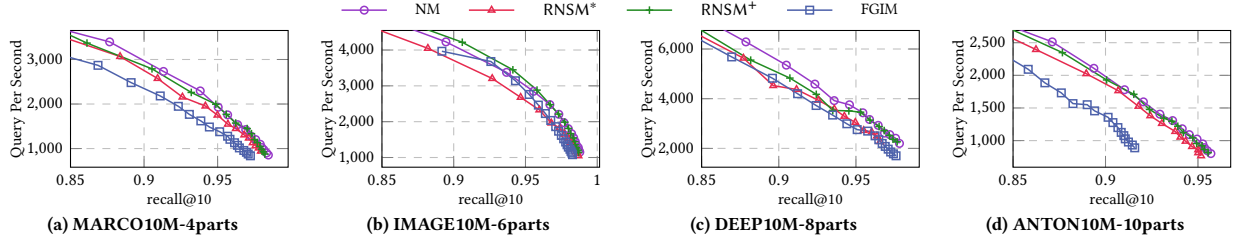


Figure 12: Recall@10-Query Per Second Tradeoff for Large Scale Dataset

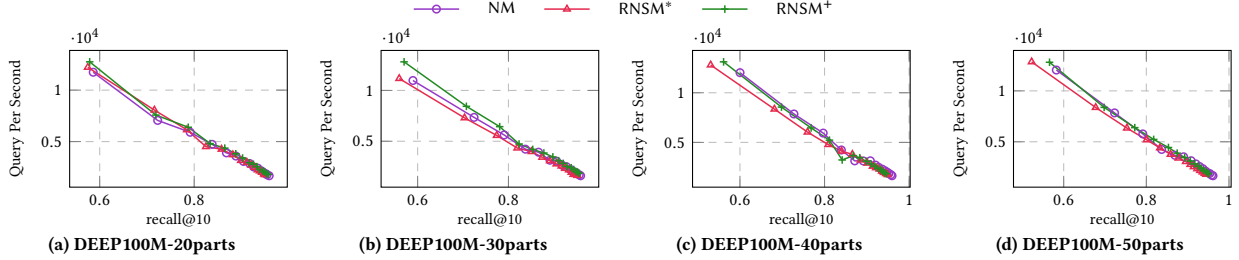


Figure 13: Recall@10-Query Per Second Tradeoff for More Data Partitions

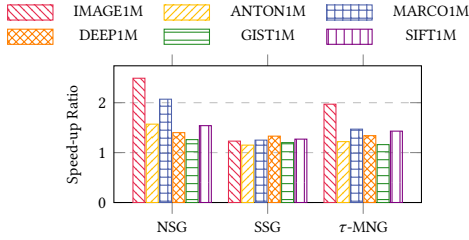


Figure 14: Speed-up Ratio Comparison of Different Methods

{20, 30, 40, 50} subsets using both random and k-means partitioning and merge them using RNSM^* and RNSM^+ , respectively. We exclude FGIM here as its theoretical guarantees do not hold when the partition number approaches the maximum degree (usually 32) of the sub-indexes. Results are shown in Fig. 11 and Fig. 13. RNSM consistently outperforms the NM baseline without compromising search performance. Notably, the efficiency advantage of our framework amplifies as the system scales: the speedup ratio grows as the number of partitions increases, demonstrating that RNSM is well-suited for massive multi-index merging scenarios.

Exp-5: Applicability Study. To demonstrate the generality of our framework, we evaluate RNSM on three additional graph-based index structures: NSG, SSG, and τ -MNG. We configure NSG and SSG following recommendations in [60] and implement τ -MNG on the NSG codebase as suggested in [44]. Fig. 14 presents the merge efficiency. RNSM achieves consistent speedups across all index types. For example, it delivers speedups of up to $2.49\times$ for NSG, $1.33\times$ for SSG, and $1.97\times$ for τ -MNG. Furthermore, Fig. 15 confirms that RNSM maintains search quality comparable to NM across these diverse structures, validating the broad applicability of our approach.

Exp-6: Effects of Reverse Neighbor. We analyze the impact of using Reverse KNNs (RKNN) for *local sliding*, a technique designed

to reduce merge costs via hub-first processing. By selecting pivots with large RKNN sets, we maximize the number of nodes processed via local sliding rather than naive search. We compare three variants: (1) NSM: random pivots, slide to KNN; (2) RNSM^- : random pivots, slide to RKNN; and (3) RNSM: pivots and followers selected by our greedy algorithm. We evaluate these strategies on SIFT1M with $k \in \{3, 5, 10, 20\}$ and report the proportion of local sliding operations in Fig. 16. Across all methods, the local sliding ratio grows significantly as k increases. For RNSM, the ratio rises from 66.2% at $k=3$ to 90.1% at $k=20$, confirming that higher connectivity naturally facilitates more local moves. RNSM consistently outperforms both NSM and RNSM^- across all k values. This demonstrates that prioritizing hubs (nodes with high reverse-neighbor cardinality) as pivots allows the algorithm to cover more points through efficient sliding, thereby minimizing expensive naive searches. Interestingly, RNSM^- underperforms compared to NSM. However, by simply introducing the sorting strategy, RNSM surpasses NSM. This indicates that while RKNN-based sliding is a powerful tool, its success is highly dependent on the order of pivot processing.

Exp-7: Effects of MOS Technique. Finally, we validate the effectiveness of the Merge Order Search (MOS) strategy for merging cluster partitions. We compare the search performance of our Merge Order Graph against a random regular graph with the same average degree. Using simple merge orders with fewer merge operations, as employed in motivations like MST, Star, and Path, fails to achieve a search recall of 0.9 and is therefore not considered in our experiment. Experiments are conducted on DEEP10M split into 10 cluster partitions. As shown in Fig. 17, the topology generated by MOS consistently yields better search performance than the random graph. This confirms that respecting the cluster structure during the merge process is critical for maintaining high retrieval accuracy.

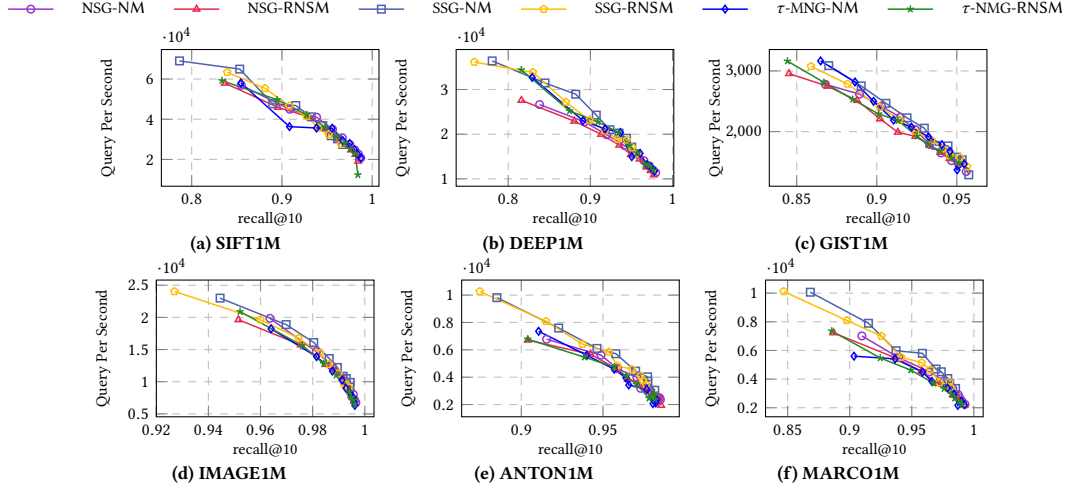


Figure 15: Recall@10-QPS Tradeoff of Different Methods

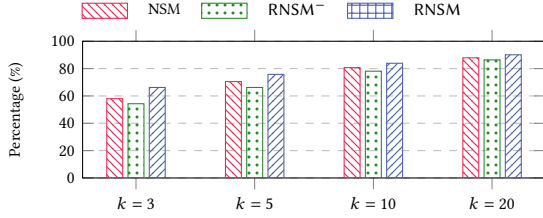


Figure 16: Comparison of Local Sliding Ratio with DEEP10M

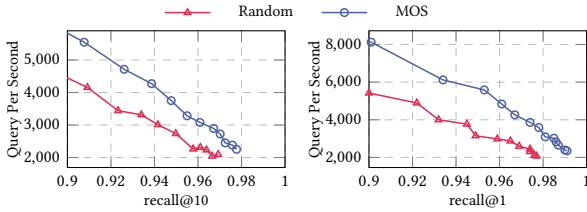


Figure 17: Comparison of Different Merge Order Selection of DEEP10M

7 RELATED WORKS

We review existing approaches for index construction and merging, evaluating them based on their scalability, computational efficiency, and memory overhead.

Similarity Join. SimJoin [66] was originally proposed to efficiently compute approximate similarity joins, outputting all pairs of vectors from two datasets whose distance falls below a specified threshold. This task is intrinsically linked to index merging: identifying cross-index KNNs between two partitions is effectively a similarity join operation restricted to the top- k nearest neighbors. Therefore, SimJoin can be viewed as an acceleration primitive for two-index merging and can be extended to handle multiple indexes through pairwise processing.

KNNG Merge. S-merge [78] proposes merging two k -Nearest Neighbor Graphs (KNNGs) by first connecting them with random edges and then iteratively refining the graph using NNDecent [15]. Multi-way Merge [75] extends this concept to merge multiple KNNGs in a single pass by cross-matching only newly added cross-index candidates. Generally, the index quality in these methods converges faster than in PG-based methods due to the batch processing nature of NNDecent [74]. However, KNNGs are not optimized for Approximate KNN (AKNN) search and typically yield inferior search performance compared to navigational graph indexes [4, 36, 60, 62]. Furthermore, these methods require loading the entire dataset into memory, which limits scalability.

Overlapping Partitioning. This strategy is often employed for billion-scale graph index construction under memory constraints (e.g., DiskANN). The dataset is partitioned into overlapping clusters via k -means, where each data point is assigned to $k_{base} \geq 2$ nearest partitions. Sub-indexes are constructed for each partition, and a compact graph is subsequently merged by concatenating the neighborhoods of identical nodes across different subgraphs. However, this approach requires clustering the dataset and building sub-indexes from scratch, making it inapplicable to scenarios where one must merge pre-existing indexes.

8 CONCLUSION

In this paper, we addressed the challenge of merging multiple proximity graph indexes to enable efficient multi-index searching. By identifying the core components of graph merging, we proposed the Reverse Neighbor Sliding Merge (RNSM) algorithm to facilitate rapid cross-index KNN identification. Furthermore, based on merge order analysis, we designed two Merge Order Selection (MOS) strategies tailored for random and cluster partitions, effectively reducing redundancy in pairwise processing. Our experimental results demonstrate that our framework significantly improves merging efficiency while preserving the superior search performance characteristic of graph indexes. Notably, our method exhibits robust scalability, performing effectively even with large-scale datasets.

REFERENCES

- [1] Cecilia Aguerrebere, Ishwar Singh Bhati, Mark Hildebrand, Mariano Tepper, and Theodore Willke. 2023. Similarity Search in the Blink of an Eye with Compressed Indices. *Proceedings of the VLDB Endowment* (2023), 3433–3446.
- [2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2015. Cache Locality Is Not Enough: High-Performance Nearest Neighbor Search with Product Quantization Fast Scan. *Proceedings of the VLDB Endowment* (2015), 288–299.
- [3] Sunil Arya and David M. Mount. 1993. Approximate Nearest Neighbor Queries in Fixed Dimensions. In *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '93)*. Society for Industrial and Applied Mathematics, USA, 271–280.
- [4] Martin Aumüller, Erik Bernhardtsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Information Systems* (2020), 101374.
- [5] Artem Babenko and Victor Lempitsky. 2015. The Inverted Multi-Index. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2015), 1247–1260.
- [6] Jonathan Bac, Evgeny M Mirkes, Alexander N Gorban, Ivan Tyukin, and Andrei Zinovyev. 2021. Scikit-dimension: a python package for intrinsic dimension estimation. *Entropy* 23, 10 (2021), 1368.
- [7] Alina Beygelzimer, Sham Kakade, and John Langford. 2006. Cover Trees for Nearest Neighbor. In *Proceedings of the 23rd International Conference on Machine Learning - ICML '06*. ACM Press, Pittsburgh, Pennsylvania, 97–104.
- [8] Patrick Chen, Wei-Cheng Chang, Jyun-Yu Jiang, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh. 2023. FINGER: Fast Inference for Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the ACM Web Conference 2023*. ACM, Austin TX USA, 3225–3235.
- [9] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. Spann: Highly-efficient billion-scale approximate nearest neighborhood search. *Advances in Neural Information Processing Systems* 34 (2021), 5199–5212.
- [10] T. Cover and P. Hart. 1967. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory* 13, 1 (1967), 21–27. <https://doi.org/10.1109/TIT.1967.1053964>
- [11] Abhinandan S. Das, Mayur Datar, Ashutosh Garg, and Shyam Rajaram. 2007. Google News Personalization: Scalable Online Collaborative Filtering. In *Proceedings of the 16th International Conference on World Wide Web (WWW '07)*. Association for Computing Machinery, New York, NY, USA, 271–280. <https://doi.org/10.1145/1242572.1242610>
- [12] Sanjoy Dasgupta and Yoav Freund. 2008. Random Projection Trees and Low Dimensional Manifolds. In *Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing*. ACM, Victoria British Columbia Canada, 537–546. <https://doi.org/10.1145/1374376.1374452>
- [13] Mayur Datar, Piotr Indyk, Nicole Immorlica, and Vahab S. Mirrokni. 2004. Locality-Sensitive Hashing Scheme Based on p-Stable Distributions. *Proceedings of the Annual ACM-SIAM Symposium on Discrete Algorithms* (2004).
- [14] Magdalen Dobson, Zheqi Shen, Guy E. Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2023. Scaling Graph-Based ANNS Algorithms to Billion-Size Datasets: A Comparative Analysis.
- [15] Wei Dong, Charikar Moses, and Kai Li. 2011. Efficient K-Nearest Neighbor Graph Construction for Generic Similarity Measures. In *Proceedings of the 20th International Conference on World Wide Web*. ACM, Hyderabad India, 577–586.
- [16] Karima Echihabi, Panagioti Fatourou, Kostas Zoumpatianos, Themis Palpanas, and Houda Benbrahim. 2022. Hercules Against Data Series Similarity Search.
- [17] Cong Fu, Changxu Wang, and Deng Cai. 2022. High Dimensional Similarity Search With Satellite System Graph: Efficiency, Scalability, and Unindexed Query Compatibility. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2022), 4139–4150.
- [18] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph. *Proceedings of the VLDB Endowment* (2019), 461–474.
- [19] Junhao Gan, Jianlin Feng, Qiong Fang, and Wilfred Ng. 2012. Locality-Sensitive Hashing Scheme Based on Dynamic Collision Counting. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. ACM, Scottsdale Arizona USA, 541–552.
- [20] Jianyang Gao, Yutong Gou, Yuxuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. 2024. Extend-RaBitQ: Practical and Asymptotically Optimal Quantization of High-Dimensional Vectors in Euclidean Space for Approximate Nearest Neighbor Search.
- [21] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: With Reliable and Efficient Distance Comparison Operations. *Proceedings of the ACM on Management of Data* (2023), 1–27.
- [22] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 3, Article 167 (May 2024), 27 pages. <https://doi.org/10.1145/3654970>
- [23] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 4 (2014), 744–755. <https://doi.org/10.1109/TPAMI.2013.240>
- [24] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. (1999), 518–529.
- [25] Yutong Gou, Jianyang Gao, Yuxuan Xu, and Cheng Long. 2024. SymphonyQG: Towards Symphonious Integration of Quantization and Graph for Approximate Nearest Neighbor Search.
- [26] Ben Harwood and Tom Drummond. 2016. FANNG: Fast Approximate Nearest Neighbour Graphs. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Las Vegas, NV, USA, 5713–5722.
- [27] A. J. Hoffman and R. R. Singleton. 1960. On Moore graphs with diameters 2 and 3. *IBM J. Res. Dev.* 4, 5 (Nov. 1960), 497–504. <https://doi.org/10.1147/rd.45.0497>
- [28] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing - STOC '98*. ACM Press, Dallas, Texas, United States, 604–613.
- [29] Piotr Indyk and Haikuo Xu. 2023. Worst-Case Performance of Popular Approximate Nearest Neighbor Search Implementations: Guarantees and Limitations.
- [30] H. Jégou, M. Douze, and C. Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2011), 117–128.
- [31] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guanhuo Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibao Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proc. ACM Manag. Data* 3, 3, Article 148 (June 2025), 26 pages. <https://doi.org/10.1145/3725399>
- [32] Liuchang Jing, Mingyu Yang, Lei Li, Jianbin Qin, and Wei Wang. 2025. *Technical Report*. https://github.com/JingLiuchang/hnsw-merge/blob/master/technical_report.pdf
- [33] Richard M Karp. 2009. Reducibility among combinatorial problems. In *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art*. Springer, 219–241.
- [34] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [35] Binhong Li, Xiao Yan, and Shangqi Lu. 2025. Fast-Convergent Proximity Graphs for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* (2025).
- [36] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2019. Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement. , 1475–1488 pages.
- [37] Yiqi Li, Sheng Wang, Zhiyu Chen, Shangfeng Chen, and Zhiyong Peng. 2024. Approximate Vector Set Search: A Bio-Inspired Approach for High-Dimensional Spaces.
- [38] Ying Liu, Dengsheng Zhang, Guojun Lu, and Wei-Ying Ma. 2007. A Survey of Content-Based Image Retrieval with High-Level Semantics. *Pattern Recognition* 40, 1 (2007), 262–282. <https://doi.org/10.1016/j.patcog.2006.04.045>
- [39] Yuriy Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate Nearest Neighbor Algorithm Based on Navigable Small World Graphs. *Information Systems* (2014), 61–68.
- [40] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2020), 824–836.
- [41] Yu A Malkov and Dmitry A Yashunin. 2020. hnswlib. <https://github.com/nmslib/hnswlib>
- [42] Gonzalo Navarro. 2002. Searching in Metric Spaces by Spatial Approximation. *The VLDB Journal The International Journal on Very Large Data Bases* (2002), 28–46.
- [43] Zhaojie Niu, Xinhui Tian, Xindong Peng, and Xing Chen. 2025. BlendHouse: A Cloud-Native Vector Database System in ByteHouse. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 4332–4345.
- [44] Yun Peng, Byron Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient Approximate Nearest Neighbor Search in Multi-dimensional Databases. *Proc. ACM Manag. Data* 1, 1, Article 54 (2023), 27 pages. <https://doi.org/10.1145/3588908>
- [45] James Philbin, Ondrej Chum, Michael Isard, Josef Sivic, and Andrew Zisserman. 2007. Object retrieval with large vocabularies and fast spatial matching. In *2007 IEEE Conference on Computer Vision and Pattern Recognition*. 1–8. <https://doi.org/10.1109/CVPR.2007.383172>
- [46] Runwen Qiu and Jing Tang. 2025. Efficient Approximate Nearest Neighbor Search via Hemi-Sphere Centroids Graph. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–26.
- [47] J. Ben Schafer, Dan Frankowski, Jon Herlocker, and Shilad Sen. 2007. Collaborative Filtering Recommender Systems. In *The Adaptive Web: Methods and Strategies of Web Personalization*, Peter Brusilovsky, Alfred Kobsa, and Wolfgang Nejdl (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 291–324. https://doi.org/10.1007/978-3-540-72079-9_9

- [48] Zhenpei Shan, Defu Zhang, and Yunqi Lei. 2025. DF-LSH: An Efficient Double Filters Locality Sensitive Hashing for Approximate Nearest Neighbor Search. *Engineering Applications of Artificial Intelligence* (2025), 110742.
- [49] Yang Shi, Yiping Sun, Jiaolong Du, Xiaocheng Zhong, Zhiyong Wang, and Yao Hu. 2025. Scalable Overload-Aware Graph-Based Index Construction for 10-Billion-Scale Vector Similarity Search.
- [50] Ziwen Song, Bin Wang, and Xiaochun Yang. 2025. Accelerating High-Dimensional ANN Search via Skipping Redundant Distance Computations. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–29.
- [51] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. *DiskANN: fast accurate billion-point nearest neighbor search on a single node*. Curran Associates Inc., Red Hook, NY, USA.
- [52] Yifang Sun, Wei Wang, Jianbin Qin, Ying Zhang, and Xuemin Lin. 2014. SRS: Solving c -Approximate Nearest Neighbor Queries in High Dimensional Euclidean Space with a Tiny Index. *Proceedings of the VLDB Endowment* (2014), 1–12.
- [53] Yufei Tao, Ke Yi, Cheng Sheng, and Panos Kalnis. 2010. Efficient and Accurate Nearest Neighbor and Closest Pair Search in High-Dimensional Space. *ACM Transactions on Database Systems* (2010), 1–46.
- [54] Yao Tian, Xi Zhao, and Xiaofang Zhou. 2024. DB-LSH 2.0: Locality-Sensitive Hashing With Query-Based Dynamic Bucketing. *IEEE Transactions on Knowledge and Data Engineering* (2024), 1000–1015.
- [55] Godfried T. Toussaint. 1980. The Relative Neighbourhood Graph of a Finite Planar Set. *Pattern Recognition* 12, 4 (Jan. 1980), 261–268. [https://doi.org/10.1016/0031-3203\(80\)90066-7](https://doi.org/10.1016/0031-3203(80)90066-7)
- [56] Sairaj Voruganti and M. Tamer Özsu. 2025. MIRAGE-ANNS: Mixed Approach Graph-based Indexing for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3, Article 188 (June 2025), 27 pages. <https://doi.org/10.1145/3725325>
- [57] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data*. ACM, Virtual Event China, 2614–2627. <https://doi.org/10.1145/3448016.3457550>
- [58] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating Graph Indexing for ANNS on Modern CPUs. *Proceedings of the ACM on Management of Data (SIGMOD)* (2025).
- [59] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proceedings of the ACM on Management of Data* (2024), 1–27.
- [60] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search.
- [61] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* (2021), 1964–1978.
- [62] Zeyu Wang, Peng Wang, Themis Palpanas, and Wei Wang. 2023. Graph- and Tree-based Indexes for High-dimensional Vector Similarity Search: Analyses, Comparisons, and Future Directions. *IEEE Data Eng. Bull.* 47 (2023), 3–21. <https://api.semanticscholar.org/CorpusID:265509541>
- [63] Jiuqi Wei, Xiaodong Lee, Zhenyu Liao, Themis Palpanas, and Botao Peng. 2025. Subspace Collision: An Efficient and Accurate Framework for High-dimensional Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* (2025), 79:1–79:29.
- [64] Jiuqi Wei, Botao Peng, Xiaodong Lee, and Themis Palpanas. 2024. DET-LSH: A Locality-Sensitive Hashing Scheme with Dynamic Encoding Tree for Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* (2024), 2241–2254.
- [65] Zekai Wu, Jiabao Jin, Peng Cheng, Xiaoyao Zhong, Lei Chen, Zhitao Shen, Jingkuan Song, Xiaofeng Cao, Heng Tao Shen, and Xuemin Lin. 2024. Fast Graph-based Indexes Merging for Approximate Nearest Neighbor Search. (2024).
- [66] Jiadong Xie, Jeffrey Xu Yu, and Yingfan Liu. 2025. Fast Approximate Similarity Join in Vector Databases. *Proceedings of the ACM on Management of Data* (2025), 1–26.
- [67] Qian Xu, Feng Zhang, Chengxi Li, Lei Cao, Zheng Chen, Jidong Zhai, and Xiaoyong Du. 2025. HARMONY: A Scalable Distributed Vector Database for High-Throughput Approximate Nearest Neighbor Search.
- [68] Ming Yang, Yuzheng Cai, and Weiguo Zheng. 2024. CSPG: Crossing Sparse Proximity Graphs for Approximate Nearest Neighbor Search. *Advances in Neural Information Processing Systems* (2024), 103076–103100.
- [69] Mingyu Yang, Wentao Li, Jiabao Jin, Xiaoyao Zhong, Xiangyu Wang, Zhitao Shen, Wei Jia, and Wei Wang. 2024. Effective and General Distance Computation for Approximate Nearest Neighbor Search.
- [70] Mingyu Yang, Wentao Li, Zhitao Shen, Chuan Xiao, and Wei Wang. 2025. ESG: Elastic Graphs for Range-Filtering Approximate k-Nearest Neighbor Search.
- [71] Mingyu Yang, Wentao Li, and Wei Wang. 2024. Fast High-dimensional Approximate Nearest Neighbor Search with Efficient Index Time and Space.
- [72] Shuo Yang, Jiadong Xie, Yingfan Liu, Jeffrey Xu Yu, Xiyue Gao, Qianru Wang, Yanguo Peng, and Jiangtao Cui. 2025. Revisiting the Index Construction of Proximity Graph-Based Approximate Nearest Neighbor Search.
- [73] Kun Yu, Jiabao Jin, Xiaoyao Zhong, Peng Cheng, Lei Chen, Zhitao Shen, Jingkuan Song, Hengtao Shen, and Xuemin Lin. 2025. Approximate Nearest Neighbor Search of Large Scale Vectors on Distributed Storage.
- [74] Cheng Zhang, Jianzhi Wang, Wan-Lei Zhao, and Shihai Xiao. 2025. Highly Efficient Disk-based Nearest Neighbor Search on Extended Neighborhood Graph. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, Padua Italy, 2513–2523.
- [75] Cheng Zhang, Wan-Lei Zhao, Shihai Xiao, Jiajie Yao, and Xuechang Zhang. 2025. Towards the Distributed Large-scale k-NN Graph Construction by Graph Merge.
- [76] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, et al. 2023. {VBASE}: Unifying online vector similarity search and relational queries via relaxed monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 377–395.
- [77] Kang Zhao, Pan Pan, Yun Zheng, Yanhao Zhang, Changxu Wang, Yingya Zhang, Yinghui Xu, and Rong Jin. 2021. Large-Scale Visual Search with Binary Distributed Graph at Alibaba.
- [78] Wan-Lei Zhao, Hui Wang, Peng-Cheng Lin, and Chong-Wah Ngo. 2022. On the Merge of k-NN Graph. *IEEE Transactions on Big Data* 8, 6 (2022), 1496–1510. <https://doi.org/10.1109/TBDDATA.2021.3101517>
- [79] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *Proceedings of the VLDB Endowment* (2023), 1979–1991.
- [80] Xiaoyao Zhong, Jiabao Jin, Peng Cheng, Mingyu Yang, Haoyang Li, Zhitao Shen, Heng Tao Shen, and Jingkuan Song. 2025. EnhanceGraph: A Continuously Enhanced Graph-based Index for High-dimensional Approximate Nearest Neighbor Search.
- [81] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, Xiangyu Wang, Zhitao Shen, Wei Jia, George Gu, Yi Xie, Xuemin Lin, Heng Tao Shen, Jingkuan Song, and Peng Cheng. 2025. VSAG: An Optimized Search Framework for Graph-based Approximate Nearest Neighbor Search.

APPENDIX

Proofs

Proof of Theorem 4.2

PROOF. We recall the Minimum Dominating Set (MDS) problem. Given an undirected graph $G = (V, E)$, the goal is to find a minimum-size subset $P \subseteq V$ such that every node $v \in V$ is either in P or adjacent to at least one node in P . The MDS problem is known to be NP-hard [33].

We prove the NP-hardness of DPS by reduction from MDS. We construct a DPS instance from an MDS instance $G = (V, E)$ by using the same graph, setting $\delta(x_i, x_j) = 1$ for all $i \neq j$, and $\gamma = |V| + 1$.

Since $\delta(x_i, x_j) = 1$ for all pairs, for any valid dominating set $P \subseteq V$, we have:

$$\mathcal{L}_{slide}(P) = \sum_{v \in V \setminus P} \min_{p \in P \cap N(v)} \delta(x_v, x_p) = |V \setminus P| = |V| - |P|$$

Therefore, the total cost is:

$$\mathcal{L}(P) = \gamma \cdot |P| + \mathcal{L}_{slide}(P) = |V| \cdot |P| + |V|$$

This shows that for any two valid dominating sets P_1 and P_2 :

$$\mathcal{L}(P_1) < \mathcal{L}(P_2) \Leftrightarrow |P_1| < |P_2|$$

Thus, minimizing $\mathcal{L}(P)$ is equivalent to minimizing $|P|$, which means the optimal solution to the constructed DPS instance is exactly the optimal solution to the original MDS instance.

Therefore, if there exists a polynomial-time algorithm for DPS, we can solve any MDS instance in polynomial time, contradicting the NP-hardness of MDS. Hence, DPS is NP-hard. $\square$

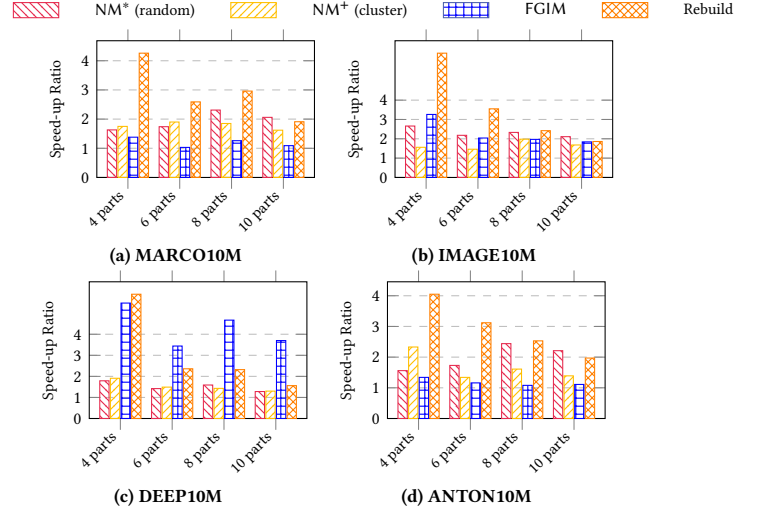


Figure 18: Speed-up Ratio Comparison across Different Partitions

Extra Experiments

Complete Results for Exp-2 This section provides the complete experimental results for Exp-2: Multiple Indexes Merging. We present the full recall@10-QPS tradeoff curves for all datasets (DEEP10M, MARCO10M, IMAGE10M, and ANTON10M) under both random and cluster partitions with varying partition numbers in Fig. 19. The results show that our RNSM and MOS framework maintains the expected search performance of graph indexes and consistently outperforms the existing method, FGIM. We also report the complete speedup ratios for all datasets in Fig. 18. Our method shows the best merge efficiency across all partition numbers and datasets, achieving up to 2.33 $\times$, 5.48 $\times$, 6.43 $\times$ acceleration compared to NM, FGIM, and BuildAsOne.

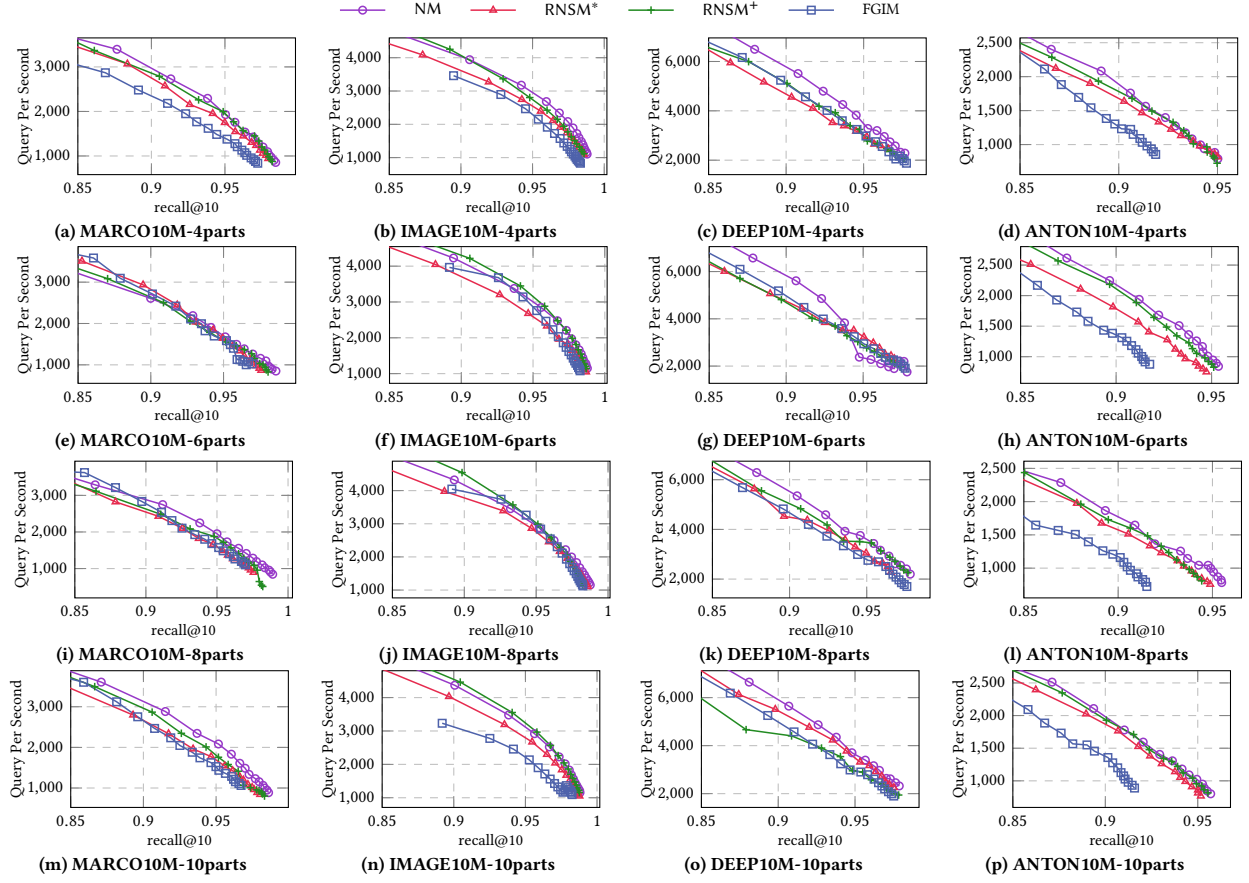


Figure 19: Recall@10-Query Per Second Tradeoff for Merge Methods